

The Most Common Topics Involving Randomization

Peter Occil

This version of the document is dated 2025-09-30.

Peter Occil

Abstract: This article goes over some of the most common topics involving randomization in programming, and serves as a guide to programmers looking to solve their randomization problems. They were based on the most commonly pointed-to questions involving randomization on a Q&A site. The topics included generating uniform random variates, unique random values, choosing one or more random items, shuffling, and querying random records from a database.

1 Introduction

This page goes over some of the most common topics involving randomization (including “random number generation”) in programming, and serves as a guide to programmers looking to solve their randomization problems.

The topics on this page were chosen based on an analysis of the *Stack Overflow* questions that other questions were most often marked as duplicates of (using the *Stack Exchange Data Explorer* query named “Most popular duplicate targets by tag”, with “random” as the TagName).

The analysis showed the following topics were among the most commonly asked:

- Generating uniform random integers in a range.
- Generating uniform random floating-point numbers in a range.
- Generating unique random integers in a range.
- Choosing an item at random from a list.
- Choosing several unique items from a list.
- Choosing items with separate probabilities.
- Choosing records at random from a database.
- Shuffling.
- Generating a random text string of characters selected from a restricted character set (such as only A to Z, a to z, 0 to 9).

Not all topics are covered above. Notably, the analysis ignores questions that were API-specific or programming-language specific, unless the underlying issue is present in multiple APIs or languages.

Another notable trend is that these topics were asked for programming languages where convenient APIs for these tasks were missing. This is why I recommend that **new programming language APIs**¹ *provide functionality covering the topics above in their standard libraries*, to ease the burden of programmers using that language.

¹https://peteroupc.github.io/random.html#Implementing_New_RNG_APIS

The following sections will detail the topics given earlier, with suggestions on how to solve them. Many of the links point to sections of my article “**Randomization and Sampling Methods**”².

The **pseudocode conventions**³ apply to this document.

All the randomization methods presented on this page assume there is a source of “truly” random numbers.

2 Contents

- **Introduction**
- **Contents**
- **Uniform Numbers in a Range**
- **Choosing Random Items**
- **Unique Integers or Items**
- **Shuffling**
- **Random Records from a Database**
- **Random Character Strings**
- **Choosing Items with Separate Probabilities**
- **Other Topics**
- **Notes**
- **License**

3 Uniform Numbers in a Range

For algorithms on generating uniform random *integers* in a range, see “**Uniform Random Integers**”⁴. It should be noted there that most pseudorandom number generators in common use output 32- or 64-bit nonnegative integers, and for JavaScript, the idiom `(Math.random() < 0.5 ? 0 : 1)` will work in many practical cases to generate bits (zeros and ones) at random. Here is a JavaScript example of generating a random integer in the interval `[*minInclusive, maxExclusive)`, using the Fast Dice Roller by J. Lumbroso (2013)⁵:

```
function randomInt(minInclusive, maxExclusive) {
  var maxInclusive = (maxExclusive - minInclusive) - 1
  if (minInclusive == maxInclusive) return minInclusive
  var x = 1
  var y = 0
  while(true) {
    x = x * 2
    var randomBit = (Math.random() < 0.5 ? 0 : 1)
    y = y * 2 + randomBit
    if(x > maxInclusive) {
      if (y <= maxInclusive) { return y + minInclusive }
      x = x - maxInclusive - 1
      y = y - maxInclusive - 1
    }
  }
}
```

²<https://peteroupc.github.io/randomfunc.html>

³<https://peteroupc.github.io/pseudocode.html>

⁴https://peteroupc.github.io/randomfunc.html#Uniform_Random_Integers

⁵Goualard F. (2020) Generating Random Floating-Point Numbers by Dividing Integers: A Case Study. In: Krzhizhanovskaya V. et al. (eds) Computational Science – ICCS 2020. ICCS 2020. Lecture Notes in Computer Science, vol 12138. Springer, Cham. https://doi.org/10.1007/978-3-030-50417-5_2

```

    }
  }
}

```

Many common programming languages have no convenient or correct way to generate numbers in a range at random. For example:

- Java’s `java.util.Random` until version 8 had methods to produce `ints` in the interval $[0, n)$ (`nextInt`), but not `longs` in that interval or integers in an arbitrary interval $[a, b)$. Additional methods named `longs` and `ints` were later provided that offer this functionality, but even so, they are often not as convenient as the existing `nextInt` method.
- JavaScript until recently has only one API that exposes a random number generator, namely `Math.random()`, and no built-in method for shuffling or producing integers at random, among other things. Naïve solutions such as `Math.floor(Math.random()*x)+y` are not guaranteed to work reliably, in part because JavaScript doesn’t require any particular implementation for `Math.random`.
- C’s `rand` function produces random integers in a predetermined range $[0, \text{RAND_MAX}]$ that is not within the application’s control. This is just one of a **host of issues with `rand`**⁶, by the way (unspecified algorithm, yet is initializable with “`srand`” for repeatability; nonthread-safety; unspecified distribution; historical implementations had weak low bits; etc.).

For algorithms on generating uniform random *floating-point numbers* in a range, see “**For Floating-Point Number Formats**”⁷. Floating-point number generation has a myriad of issues not present with integer generation. For example, no computer can choose from all real numbers between two others, since there are infinitely many of them, and also, naïvely multiplying or dividing an integer by a constant (for example, `Math.random()*x` in JavaScript) will necessarily miss many representable floating-point numbers (for details, see Goualard (2020)⁸).

4 Choosing Random Items

In general, choosing a random item from a list is trivial: choose a random integer in $[0, n)$, where `n` is the size of the list, then take the item at the chosen position. The previous section already discussed how to generate a random integer.

However, if the number of items is not known in advance, then a technique called *reservoir sampling* can be used to choose one or more items at random. Here is how to implement reservoir sampling.

1. Set `N` to 1.
2. If no items remain, return the last chosen item. Otherwise, take the next item and choose it with probability $1/N$.
3. Add 1 to `N` and go to step 2.

See “**Pseudocode for Random Sampling**”⁹ for an algorithm for reservoir sampling.

⁶<https://stackoverflow.com/questions/52869166/why-is-the-use-of-rand-considered-bad/52881465#52881465>

⁷https://peteroupc.github.io/randomfunc.html#For_Floating_Point_Number_Formats

⁸Goualard F. (2020) Generating Random Floating-Point Numbers by Dividing Integers: A Case Study. In: Krzhizhanovskaya V. et al. (eds) Computational Science – ICCS 2020. ICCS 2020. Lecture Notes in Computer Science, vol 12138. Springer, Cham. https://doi.org/10.1007/978-3-030-50417-5_2

⁹https://peteroupc.github.io/randomfunc.html#Pseudocode_for_Random_Sampling

5 Unique Integers or Items

Generating unique random integers or items is also known as sampling *without replacement*, *without repetition*, or *without duplicates*.

There are many ways to generate unique items, depending on the number of items to choose, the number of items to choose *from*, and so on, and they have different tradeoffs in terms of time and memory requirements. See “**Sampling Without Replacement: Choosing Several Unique Items**¹⁰” for advice.

Some applications require generating unique values that identify something, such as database records, user accounts, and so on. However, there are certain things to keep in mind when generating unique values for this purpose; see “**Unique Random Identifiers**¹¹” for more information.

6 Shuffling

An algorithm to randomize (*shuffle*) the order of a list is given in “**Shuffling**”¹². It should be noted that the algorithm is easy to implement incorrectly. Also, the choice of random number generator is important when it comes to shuffling; see my **RNG recommendation document on shuffling**¹³.

7 Random Records from a Database

Querying random records (*rows*) from a database usually involves the database language SQL. However, SQL is implemented very differently in practice between database management systems (DBMSs), so that even trivial SQL statements are not guaranteed to work the same from one DBMS to another. Moreover, SQL has no loops, no branches, and no standard way to produce randomly generated or pseudorandom numbers. Thus, the correct way to query random records from a database will vary from DBMS to DBMS.

With that said, the following specific situations tend to come up in random record queries.

- Querying one random record from a database.
- Querying a specified number of random records from a database.
- Querying one or more records each with a probability proportional to its weight. Very generally, this can be done by giving the table a column where each entry is a number generated as follows: $\ln(R) / W$ (where W is the record’s weight greater than 0, itself its own column, and R is a per-record uniform random variate in the open interval $(0, 1)$) (see also (Arratia 2002)¹⁴), then taking the records with the highest values of that column, but the efficiency of this technique depends on the DBMS.

8 Random Character Strings

Many applications need to generate a random string whose characters are chosen from a restricted set of characters. Popular choices include so-called *alphanumeric strings*, where the restricted character set is $A \cup Z$, a to z , 0 to 9 . An algorithm for generating random strings is given in “**Random Character Strings**¹⁵”.

However, the following are some of the many considerations involving random string generation:

¹⁰https://peteroupc.github.io/randomfunc.html#Sampling_Without_Replacement_Choosing_Several_Unique_Items

¹¹https://peteroupc.github.io/random.html#Unique_Random_Identifiers

¹²<https://peteroupc.github.io/randomfunc.html#Shuffling>

¹³<https://peteroupc.github.io/random.html#Shuffling>

¹⁴Arratia, R., “On the amount of dependence in the prime factorization of a uniform random integer”, *Contemporary Combinatorics* 10(29), 91, 2002.

¹⁵https://peteroupc.github.io/randomfunc.html#Random_Character_Strings

- If the string needs to be typed in by customers, or to be memorable, it may be important to choose a character set carefully or **allow typing mistakes to be detected**¹⁶.
- If the string identifies something, the application may require strings it generates to be unique; see **Unique Random Identifiers**¹⁷ for considerations.
- If the string is a secret value of any kind, including a password, a bearer credential, a session identifier, a nonce, a “confirmation code”, a “verification code”, or a “forgot-password” code, then the string has to be generated using a **cryptographic RNG**¹⁸ (such as the `secrets` module in Python or the `random_bytes` function in PHP).

9 Choosing Items with Separate Probabilities

Weighted choice (also known as a *categorical distribution*) is a random choice of items, where each item has a *weight* and is chosen with a probability proportional to its weight.

For algorithms on weighted choice, see “**Weighted Choice With Replacement**¹⁹”, which covers choices in which items are taken and put back.

The pseudocode shown there is a straightforward way to implement weighted choice, but there are other alternatives (many of which are implemented in **Python sample code**²⁰). They include rejection sampling, Vose’s version of the alias method (`VoseAlias`; see “**Darts, Dice, and Coins: Sampling from a Discrete Distribution**²¹” by Keith Schwarz for more information), and the Fast Loaded Dice Roller (`FastLoadedDiceRoller`) (Saad et al. 2020)²².

Weighted choice *without replacement* is a choice where each item can be chosen no more than once. If the weights have the property that higher-weighted items have a greater chance of appearing first, then:

- The simplest way to implement this is to use weighted choice with replacement, except that after an index is chosen, that index’s weight is set to 0 to keep the index from being chosen again.
- Other options are given in “**Weighted Choice Without Replacement**²³”.

However, these methods do not necessarily ensure that a random sample of n items will include a given item with probability proportional to that item’s weight. This is a similar problem that is not solved by these methods; for that problem, see “**Algorithms of sampling with equal or unequal probabilities**²⁴”.

Note that choosing *true* with a given probability, or *false* otherwise, is a special case of weighted sampling involving two items (also known as a *Bernoulli trial*). But there are much simpler ways of choosing *true* or *false* this way; see “**Boolean (True/False) Conditions**²⁵”. Perhaps the most practical is the idiom `RNDINTEXC(Y) < X`, which chooses *true* with probability X/Y , *false* otherwise.

¹⁶<https://espadrine.github.io/blog/posts/a-base32-checksum.html>

¹⁷https://peteroupc.github.io/random.html#Unique_Random_Identifiers

¹⁸https://peteroupc.github.io/random.html#Existing_RNG_APIs_in_Programming_Languages

¹⁹https://peteroupc.github.io/randomfunc.html#Weighted_Choice_With_Replacement

²⁰<https://peteroupc.github.io/randomgen.zip>

²¹<https://www.keithschwarz.com/darts-dice-coins/>

²²Saad, F.A., Freer C.E., et al. “The Fast Loaded Dice Roller: A Near-Optimal Exact Sampler for Discrete Probability Distributions”, in *AISTATS 2020: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics, Proceedings of Machine Learning Research* 108, Palermo, Sicily, Italy, 2020.

²³https://peteroupc.github.io/randomfunc.html#Weighted_Choice_Without_Replacement

²⁴https://www.eustat.eus/productosServicios/52.1_Unequal_prob_sampling.pdf

²⁵https://peteroupc.github.io/randomfunc.html#Boolean_True_False_Conditions

10 Other Topics

Other topics showed up in the analysis, and it's worth mentioning them here. These topics included:

- Generating a random *derangement*, or a random shuffle where every item moves to a different position (see “**Shuffling**”²⁶; see also [questions/25200220](#)).
- Generating a number that follows the **normal distribution**²⁷.
- Generating a number that follows an **arbitrary distribution**²⁸.
- **Random colors**²⁹.
- Randomly generating **numbers with a given sum**³⁰.
- Randomly generating **dates and times**³¹.
- Stratified sampling (per-group sampling).
- Generating a **random point inside a circle**³².

11 Notes

12 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under **Creative Commons Zero**³³.

²⁶<https://peteroupc.github.io/randomfunc.html#Shuffling>

²⁷https://peteroupc.github.io/randomnotes.html#Normal_Gaussian_Distribution

²⁸https://peteroupc.github.io/randomfunc.html#Random_Numbers_from_an_Arbitrary_Distribution

²⁹https://peteroupc.github.io/colorgen.html#Generating_a_Random_Color

³⁰https://peteroupc.github.io/randomfunc.html#Random_Integers_with_a_Given_Positive_Sum

³¹https://peteroupc.github.io/randomfunc.html#Random_Dates_and_Times

³²https://peteroupc.github.io/randomfunc.html#Random_Points_Inside_a_Ball_Shell_or_Cone

³³<https://creativecommons.org/publicdomain/zero/1.0/>